Match Situation Search Engine — Project Spec

Sep 25, 2026 · @Iqbal Hussein

Overview

A web app where a user sketches a soccer situation on a pitch and gets back the most similar real moments from professional matches, plus stats on what those moments led to (shot rate, xG, turnover rate), each shown against a baseline. It's a portfolio project for SWE roles, so the engineering is the point and the analysis is a feature.

Goals

- Query-by-sketch search over positional data from real matches, with p95 latency under 150ms end to end.
- Outcome stats for each query, with sample size, confidence intervals and a baseline comparison.
- Measured search quality: recall against exact search, and precision on a hand-labelled set of queries.
- Production-grade engineering: CI/CD, layered tests, containerized deployment, a public demo, and a design write-up.

Success criteria

- Deployed demo link that works cold, on a free or cheap tier.
- A benchmark report in the repo that anyone can rerun with one command.
- A README and blog post that explain two or three real design trade-offs.

Data sources and licensing

The only data source is StatsBomb Open Data, a free GitHub repo of JSON files. It holds two layers:

Layer	What it has	Role here
Events	Every on-ball action (pass, carry, shot, pressure) with location, player, team, timestamp, and possession ID. Shots carry StatsBomb xG.	Outcome labels, possession chains, filters
360 freeze frames	For each event, the positions of visible players, flagged teammate or opponent, plus the camera's visible area polygon	The searchable "situations"

Coverage: 360 data exists only for some competitions, including recent men's Euros and World Cup and some club seasons. Check the repo's `three-sixty` folder for the current list before planning around it. The events layer covers far more matches.

Licensing: the data is free for non-commercial use with attribution. Credit StatsBomb in the app footer and README and follow their current terms. Don't charge for the app.

Known limitations, stated up front in the README

- Freeze frames only show players inside the broadcast camera view. Some players are simply missing.
- Coverage is skewed toward major tournaments and a few famous teams. Stats describe that sample, not football in general.
- Coordinates are on StatsBomb's 120 × 80 pitch grid.
- Record real dataset counts (matches, events, frames) after ingestion. Resume numbers must come from those counts, not from estimates.

Architecture

The system splits into an offline batch side, which does all the heavy work once, and a thin online side that only encodes a sketch, searches and aggregates. That split is what keeps queries fast.

```
flowchart LR
  A[StatsBomb JSON] --> B[Ingest + normalize]
  B --> C[(Postgres: events, frames)]
  C --> D[Outcome labeller]
  C --> E[Frame encoder]
  D --> F[(Outcome labels)]
  E --> G[(Vector index)]
  H[React pitch editor] --> I[Query API]
  I --> E2[Encode sketch]
  E2 --> G
  G --> J[Dedup + filter]
  J --> K[Aggregate stats]
  F --> K
  K --> H
```

Component	Choice	Why
Batch pipeline	Python, pandas or Polars	Best tooling for JSON parsing and numeric work
Storage	Postgres 16	One store for relational data and vectors
Vector search	pgvector (HNSW) first; FAISS as a benchmarked alternative	Comparing the two is itself a benchmark result
API	FastAPI with Pydantic models	Typed contracts, auto-generated OpenAPI docs
Frontend	React + TypeScript, SVG pitch	Drag-and-drop editing without a canvas library
Packaging	Docker Compose locally, one container per service	Same setup in dev, CI and production

The frame encoder is one shared module used by both the batch job and the API. If the two sides encode differently, search silently breaks, so a test pins them together (see Testing).

Data pipeline and schema

Ingestion is an idempotent batch job: rerunning it on the same input produces the same database, so it's safe to run again after any failure.

Pipeline steps

1. **Fetch:** clone or sparse-checkout the open-data repo at a pinned commit, so every build uses the same data.
2. **Parse:** load competitions, matches, events and 360 files. Validate each record against a schema and send malformed ones to a reject log, not a crash.
3. **Normalize:** transform every frame so the team in possession always attacks left to right. Optionally mirror top to bottom, so a left-wing situation and a right-wing one can match (store a `mirrored` flag).
4. **Load:** bulk-insert with `COPY`, in one transaction per match.
5. **Derive:** run the outcome labeller and the frame encoder (next sections), then build the vector index.

Core tables

Table	Key columns
`matches`	`match_id`, competition, season, home and away team, date
`events`	`event_id` (UUID), `match_id`, `possession_id`, `team_id`, `player_id`, `type`, `x`, `y`, `ts`, `xg` (shots only)
`frames`	`event_id`, `match_id`, `possession_id`, `players` (JSONB: x, y, teammate flag, keeper flag), `visible_area`, `ball_x`, `ball_y`, `mirrored`
`frame_vectors`	`event_id`, `encoding_version`, `embedding` (vector)
`outcomes`	`event_id`, `shot`, `xg_next`, `goal`, `turnover`, `box_entry`, `seconds_to_end`
`zone_baselines`	`zone_id`, `encoding_version`, averages for each outcome

`encoding_version` lets several encodings live side by side, which the benchmarks need. Add indexes on `possession_id` and on the filter columns (competition, team).

Situation encoding

The encoder turns a frame, or a user's sketch, into a fixed-length vector where "nearby vectors" means "similar situations." This is the project's central design problem. Build at least three encodings and let the benchmarks pick one.

Requirements

- **Order doesn't matter:** swapping which defender is which must not change the vector.
- **Variable player count:** frames show anywhere from a handful of players to all 22, and sketches usually show fewer.
- **Ball-relative:** the area around the ball matters far more than a defender 60m away.
- **Deterministic and fast:** under 1ms per frame, since it also runs inside the API.

Candidate encodings

ID	Encoding	Strengths	Weaknesses
E0	Occupancy grid: pitch split into cells (e.g. 12 × 8), one channel for attackers and one for defenders, Gaussian-smoothed, plus ball position	Order-independent by design, simple, the baseline	Blurry; treats all areas of the pitch as equally important
E1	Ball-centred polar grid: rings and sectors around the ball, finer close to it	Focuses on what matters locally; works across pitch zones	Loses absolute position unless zone is added as a feature or filter

ID	Encoding	Strengths	Weaknesses
E2	Tactical features: attackers vs defenders ahead of the ball, nearest-defender distance, defensive line height, team width and compactness, open passing lanes	Interpretable; easy to explain in the UI	Hand-picked, so it can miss patterns you didn't think of
E3 (stretch)	Learned set encoder (Deep Sets or a small transformer) trained contrastively on augmented frames	Can learn what similarity means	Needs training infrastructure; harder to justify without the eval set

Handling missing players: a blank area in a freeze frame can mean "no one there" or "off camera." Use the visible-area polygon to mask off-camera cells as unknown rather than empty. For sketches, treat only the area the user drew near as known. Document this choice; interviewers will ask about it.

Normalize and weight vector components so distance behaves sensibly, and pick cosine or L2 distance per encoding based on the benchmarks.

Search and indexing

Search is approximate nearest-neighbour (ANN) lookup over the vectors, followed by filtering and deduplication. Exact brute-force search stays in the codebase as the reference implementation.

Query flow

1. Encode the sketch with the shared encoder.
2. Ask the index for more candidates than needed (e.g. 5× the requested k), because dedup and filters will remove some.
3. Apply filters: competition, team, game state (winning, drawing, losing), pitch zone of the ball.
4. **Deduplicate by possession:** keep only the closest frame from each possession. Consecutive frames in one possession look almost identical, so without this, 50 results can be 6 plays repeated.

5. If fewer than k results remain, query again with a larger candidate count.
6. Return the top k with distance, match info and event IDs for playback.

Index choices to benchmark

Option	Notes
Exact scan (NumPy or SQL)	Ground truth for recall; fine at this data size, which makes it a fair baseline
pgvector HNSW	Default. Tune `m`, `ef_construction` and `ef_search`
pgvector IVFFlat	Faster to build, usually lower recall; tune `lists` and `probes`
FAISS in-process	Likely fastest; costs a separate index to keep in sync with Postgres

Filtering interacts badly with ANN: a strict filter can leave very few candidates. Measure recall with and without filters, and fall back to exact search inside the filtered subset when it's small. That fallback is a good design detail to write up.

Natural-language filters

Users narrow any query in two ways that stay in sync: **dropdowns** for competition and season, listing only what the dataset covers, and a **text box** for everything else, such as "Leverkusen only", "when losing in the last 15 minutes" or "Euro 2024 knockouts". Both produce the same structured filters the search already supports; free text never reaches the database.

Supported filter types

Filter	Example phrases	Maps to
Competition	"at the World Cup", "Bundesliga", "the Euros"	`competition_ids`
Season or edition	"2022/23", "Euro 2024", "last season"	`season_ids`

Filter	Example phrases	Maps to
Team	"Arsenal", "when Spain attack", "against Morocco"	`team_ids` + role (attacking or defending)
Player	"involving Wirtz", "Mbappé on the ball"	`player_ids` + role
Game state	"when losing", "level", "protecting a lead"	`game_state`
Time	"second half", "after the 75th minute", "stoppage time"	`minute_range`
Stage	"knockout rounds", "group stage", "final"	`stage`

How parsing works

1. **Catalogue:** build an entity table at ingestion from the data itself: every competition, season, team and player, each with aliases ("EPL", "Prem", "Premier League"; "Bayer Leverkusen", "Leverkusen", "B04"). Aliases live in a checked-in YAML file plus names from the data.
2. **Rule-based parser (primary):** normalize the text (lowercase, strip accents), match entities with a fuzzy-matching library such as RapidFuzz above a score threshold, and handle game state, time and stage with a small set of phrase patterns. It's fast, deterministic and easy to test.
3. **LLM fallback (optional):** only when rules find nothing, send the text and the catalogue's valid options to an LLM with a strict JSON schema for output. Validate every returned ID against the catalogue and drop anything unknown. Cache results by normalized text.
4. **Show the result:** display parsed filters as removable chips under the text box ("Competition: Premier League ×"), so users can see and correct what was understood. Unmatched words are shown too: "Didn't understand: 'rainy'".

Competition and season dropdowns

- Filled from `GET /v1/filters`, which returns only competition and season pairs that have 360 frames, with the frame count for each ("Euro 2024 · 51 matches · [N] frames").
- Two linked dropdowns: choosing a competition limits the season list to that competition's covered seasons. Both allow multiple selections and default to "All".

- Text and dropdowns share one filter state. Typing "at the World Cup" selects that competition in the dropdown, and changing the dropdown updates the chips. If the two conflict, the most recent change wins.
- If the text names a real competition with no 360 data, the dropdown stays unchanged and a notice explains why ("No positional data for that competition"), pointing to the dropdown for what's available.
- The list is generated from the data at ingestion, never hard-coded, so it updates when StatsBomb adds coverage.

API

- `POST /v1/filters/parse`: text in; structured filters, chips, unmatched words and coverage warnings out. The frontend calls it as the user types (debounced about 300ms).
- `POST /v1/search` also accepts `filter_text`. When both `filter_text` and `filters` are sent, explicit `filters` win on conflicts.

Testing and evaluation

- A labelled test file of about 150 phrases with expected filters, run in CI. Report exact-match accuracy and per-field precision and recall, separately for rules only and rules plus LLM.
- Property tests: parsing is deterministic for the rule path; every returned ID exists in the catalogue; adding irrelevant words never adds a filter.
- Adversarial cases: ambiguous names ("United", "Real"), typos, filters that contradict each other ("Spain vs Spain"), and prompt-injection text sent to the LLM path.
- Latency target: under 5ms on the rule path; LLM path reported separately and cached.

Schedule this with the frontend work in weeks 7–8, after the structured filters already work.

Outcome stats engine

Every search result also answers "what usually happens next?" Outcomes are labelled once in batch, so a query only aggregates labels it already has, which should add under 10ms.

Labelling (batch)

For each frame, scan forward through the events of the same possession until the possession ends or a fixed window runs out (default 15 seconds, configurable). Record:

Label	Definition
`shot`	Did the team in possession shoot?
`xg_next`	Total StatsBomb xG of shots in the window (0 if none)
`goal`	Did any shot in the window score?
`turnover`	Did possession change without a shot?
`box_entry`	Did the ball reach the penalty area?
`seconds_to_end`	Time until the shot, turnover or window end

Edge cases to decide and test: the half ending mid-window, set pieces, fouls won, own goals, and possession IDs that stay the same through a brief loose ball.

Aggregation (query time)

- Weight each neighbour by similarity, e.g. `w = 1 / (1 + distance)` or a Gaussian kernel. Unweighted averages are also shown.
- **Shot rate, turnover rate and box-entry rate:** weighted proportions with a Wilson interval.
- **Average xG:** weighted mean with a bootstrap 95% interval (1,000 resamples, precomputed seeds for reproducibility).
- **Goal rate:** shown as secondary. Goals are rare, so rates from a few dozen plays are mostly noise; lead with xG.
- Always show **n** (distinct possessions). Below n = 30, mark the stats as low confidence in the UI.

Baseline comparison

Precompute each outcome's average for all frames with the ball in the same zone (e.g. an 18-zone grid) and show the result as a ratio: "xG 1.8× the average from this zone." A raw number with nothing to compare against says very little.

Stretch: what-if view. Drag one player and re-run the query live, showing how the expected outcomes shift. Build only after the core engine is solid.

API design

A small REST API with typed request and response models; FastAPI generates the OpenAPI docs from them.

Method	Path	Purpose
POST	`/v1/search`	Sketch in, top-k similar situations plus aggregated stats out
GET	`/v1/situations/{event_id}`	One frame with match context
GET	`/v1/situations/{event_id}/sequence`	The events that followed, for playback
GET	`/v1/filters`	Available competitions, teams, seasons
GET	`/healthz`, `/readyz`	Liveness, and readiness (database and index loaded)
GET	`/metrics`	Prometheus metrics

Search request

```
{
  "ball": {"x": 84, "y": 22},
  "attackers": [{"x": 86, "y": 20}, {"x": 95, "y": 35}],
  "defenders": [{"x": 92, "y": 25}, {"x": 100, "y": 30}],
  "k": 25,
  "filters": {"competition_ids": [55], "game_state": "any"},
  "options": {"mirror": true, "encoding": "E1", "window_s": 15}
}
```

Search response (abbreviated)

```
{
  "results": [{"event_id": "…", "distance": 0.12, "match": "…", "minute": 63,
"outcome": {"shot": true, "xg_next": 0.21}}],
  "stats": {
    "n": 25,
    "shot_rate": {"value": 0.31, "ci95": [0.17, 0.50]},
    "xg_mean": {"value": 0.09, "ci95": [0.05, 0.14], "vs_zone_baseline":
1.8},
    "low_confidence": false
  },
  "timing_ms": {"encode": 0.4, "search": 18, "aggregate": 3}
}
```

Validate input bounds (coordinates inside 120 × 80, at most 11 per side, k up to 100). Returning `timing_ms` helps the benchmarks and debugging, and it's a nice touch in the UI. Add simple rate limiting and cache repeated queries in memory by a hash of the request.

Frontend

One page with three panels: sketch on the left, results and stats on the right, playback below. The demo has to make sense to a recruiter in 30 seconds without instructions.

- **Pitch editor:** an SVG pitch in StatsBomb coordinates. Click to add attackers, defenders and the ball; drag to move; right-click or long-press to delete. Include presets ("3v2 counter", "high press", "low block") so first-time visitors have something to click immediately.
- **Results list:** match, minute, similarity score, and outcome badges (shot, goal, turnover). Hovering a result overlays that frame on the sketch.
- **Stats panel:** shot rate, average xG, turnover rate and box-entry rate, each with its interval shown as a bar, the zone-baseline ratio, and n. Low-confidence results are greyed out with a short explanation.
- **Playback:** animate the ball and the event sequence after a selected frame. Freeze frames only exist at events, so animate the ball path and draw players at the start frame rather than inventing movement.
- **URL state:** encode the sketch in the URL, so any query is a shareable link. This makes the README and blog post much better.
- **Accessibility and mobile:** keyboard placement of players, and a pitch that scales down to phone width.

Filter bar: sits above the results, with competition and season dropdowns (covered data only) next to a free-text filter box, and parsed filters shown as removable chips. See Natural-language filters.

Debounce queries while dragging (about 150ms) so the what-if stretch goal works without flooding the API.

Testing strategy

Tests are layered so each type of bug gets caught at the cheapest level. The whole suite runs in CI on every pull request, apart from the slow tier, which runs nightly.

Layer	What it checks	Tools	Examples
Unit	Pure functions in isolation	pytest, Vitest	Zone assignment, Wilson interval against known values, request validation
Property-based	Invariants over thousands of random inputs	Hypothesis	See list below
Golden files	Output hasn't changed unexpectedly	pytest + snapshot files	Labels and vectors for 3 pinned matches match stored results
Integration	Pipeline and API against a real Postgres	pytest + Testcontainers	Ingest one match, search for one of its frames, get it back at distance 0
Contract	Frontend and API agree on shapes	OpenAPI-generated TS types, type check in CI	A renamed field fails the build
End-to-end	The demo works in a browser	Playwright	Load a preset, run a search, see results and stats
Performance	Latency hasn't regressed	Benchmark script in CI (see next section)	Fail if p95 rises more than 20% vs main

Property-based tests (the ones worth writing about)

- Normalizing twice equals normalizing once; mirroring twice is the identity.
- Shuffling the order of players never changes the vector.
- A frame searched against the index returns itself first.
- Approximate search results are a subset of what an exact scan would rank within the top few hundred.
- Every weighted rate is between 0 and 1, and every interval contains its point estimate.
- After dedup, no two results share a `possession_id`.

Other practices

- One shared encoder module, plus a test asserting the batch and API paths produce identical vectors for the same frame.
- A small fixture dataset (a few matches) checked into the repo, so tests never need the network.
- Coverage target around 85% on the core packages (encoder, labeller, stats). Don't chase 100% on glue code.
- Linting and types: ruff and mypy (strict on core packages), ESLint and `tsc --noEmit`, all enforced in CI.

Benchmarking and evaluation

There are two separate questions: is it fast (systems benchmarks), and is it good (retrieval quality)? Both run from one command (`make bench`), write a Markdown report with charts to the repo, and record the hardware and commit they ran on.

Systems benchmarks

Metric	How	Target
Query latency p50, p95, p99	Replay 1,000 queries (real frames plus random sketches) against the running API	p95 under 150ms end to end
Latency breakdown	Encode, search, dedup, aggregate, from `timing_ms`	Aggregate step under 10ms
Throughput	Load test with k6 or Locust at rising concurrency until p95 breaks target	Report the knee of the curve

Metric	How	Target
Recall@k	ANN top-k vs exact top-k, for k = 10 and 50, with and without filters	0.95 or higher at chosen settings
Recall vs latency curve	Sweep `ef_search` (HNSW) and `probes` (IVFFlat)	One chart per index type
Index build time and size	Time and disk for each index type	Report only
Ingest throughput	Frames per second through the full pipeline	Report only

Retrieval quality

1. **Hand-labelled evaluation set:** about 200 query frames. For each, label the top 10 results from a pooled set (results from all encodings, shuffled, with the encoding hidden) as similar or not. Blind labelling keeps you from favouring your own design.
2. **Metrics:** precision@10 and nDCG@10 for each encoding (E0 to E3).
3. **Automatic sanity check:** perturb a real frame slightly (jitter each player by 1–2m); the original should come back in the top 5 for nearly all cases.
4. **Outcome usefulness (optional):** check whether neighbour-based xG predicts the true xG of held-out frames better than the zone baseline. This shows that the stats mean something, not just the search.

Rules for honest numbers

- Warm up before measuring and report medians over several runs.
- Keep the hand-labelled set separate from anything you used to tune encoders or weights.
- Every resume number must trace back to a committed report.

Deployment, CI/CD and observability

The goal is a demo that's always up, cheap to run, and deployed automatically from `main`.

CI pipeline (GitHub Actions)

1. Lint and type-check (Python and TypeScript).
2. Unit and property tests.

3. Integration tests against Postgres with pgvector, via Testcontainers or a service container.
4. Build Docker images; Playwright end-to-end tests against the composed stack.
5. Short performance check against the fixture dataset; fail on a large p95 regression.
6. On merge to `main`: push images and deploy.

Hosting

- API: a small container host (Fly.io, Railway, Render or similar), one instance.
- Database: managed Postgres with pgvector, or Postgres in a container with a volume. Check free-tier limits against the real index size before choosing.
- Frontend: static hosting (Vercel, Netlify or Cloudflare Pages).
- The database is built offline by the batch job and restored from a dump, so production never runs ingestion.

Observability

- Structured JSON logs with a request ID on every line.
- Prometheus metrics: request latency histogram, per-stage timings, cache hit rate, error count.
- One small Grafana dashboard (Grafana Cloud's free tier works). Put a screenshot in the README.
- An uptime check on `/readyz`.

Operational details worth having

- Config through environment variables; secrets never in the repo.
- Database migrations with Alembic.
- A `make up` that brings the full stack up locally with a sample dataset in one command.

Milestones

About 10 weeks part-time alongside school. Each milestone ends with something demoable, so if you stop early, you still have a finished project.

Weeks	Milestone	Done when
1–2	Pipeline and exact search	Data ingested and normalized; a CLI returns nearest frames by brute force; CI running with unit and property tests
3–4	Encodings, index, API	E0–E2 built; pgvector HNSW index; `/v1/search` live locally; first recall and latency report
5–6	Outcome stats	Labeller with golden tests; aggregation with intervals, dedup and zone baselines in the API
7–8	Frontend	Pitch editor, presets, results, stats panel, playback, shareable URLs; Playwright tests
9	Evaluation	200-query hand-labelled set; precision and nDCG per encoding; full benchmark report
10	Ship	Deployed with monitoring; README with architecture, benchmarks and limitations; blog post published
Later	Stretch	E3 learned encoder, FAISS comparison, what-if view

Build the evaluation set earlier if you can (a few queries a week from week 4). It's tedious, and it's the step most likely to get skipped.

Risks and open questions

Risk	Impact	Mitigation
Search results don't look similar to a human	Stats become meaningless; the demo falls flat	Evaluate early with a small labelled set; try the ball-centred encoding before anything fancy
Too little 360 data for niche sketches	Low n, wide intervals	Mirroring doubles the pool; show n honestly; relax filters automatically with a notice

Risk	Impact	Mitigation
Partial camera view distorts matches	Missing players read as open space	Mask off-camera areas using the visible-area polygon
Free hosting can't hold the index in memory	Slow or failing demo	Measure index size in week 4; reduce vector size or use IVFFlat if needed
Scope creep (learned encoder, what-if view)	Nothing ships	Stretch items only after week 10 is done
StatsBomb terms or data layout change	Build breaks or use is restricted	Pin the data commit; re-read the terms before launch

Open questions

- Language for the API: Python keeps one language across the stack; Go shows range and may be faster. Decide before week 3.
- Outcome window: a fixed 15 seconds vs "until possession ends." Try both and pick with the xG-prediction check.
- Whether to show goal rate at all, given how noisy it is.

Resume bullets and write-up

Draft bullets below. Fill in the brackets only with numbers from the committed benchmark report.

- Built a query-by-sketch search engine over [N] positional frames from professional soccer matches. p95 latency of [X]ms with [Y]% recall against exact search (Python, Postgres/pgvector, React/TypeScript).
- Designed an order-independent encoding for variable-size player formations; improved precision@10 by [Z]% over a grid baseline on a blind, hand-labelled 200-query evaluation set.
- Built a batch outcome-labelling pipeline over [M] possessions, serving shot rate and xG with confidence intervals, possession-level deduplication and zone baselines in under [T]ms per query.
- Shipped with CI/CD, property-based and end-to-end tests, containerized deployment and Prometheus/Grafana monitoring.

On a one-page resume, pick the two or three strongest. Put the demo link and GitHub link on the project heading line.

README structure: a GIF of the demo at the top, then a one-paragraph pitch, an architecture diagram, benchmark tables, a limitations section, and a one-command local setup.

Blog post: focus on one or two design problems, not a tour. Good candidates are "why a naive search returns the same play 8 times" (dedup) and "how do you compare formations with different numbers of players?" (encoding). Include real charts from the benchmark report, and post it to LinkedIn and soccer-analytics communities.